# Introduction to Pandas

## Understanding tabular data

CS 22B, Spring 2026

Module 05: Data Exploration

Presented by Jessica Huynh-Westfall, Ph.D.

02

# AGENDA

- Tabular data
- Intro to Pandas
- Selections and indexing
- Working with NA values

03

# Chapter 14: Exceptions

Quick Python

# Class exercise

04

## Scenario:

You are working on a file in your word processor. You want to write files to your disk, however you may run out of disk space before all of the data is written.

*How do you handle this problem?*

# pandas library

05


- Tool built on top of Python program
- Offers data structure and operation for manipulating tabular data
- Functions for analyzing, cleaning, manipulating, and exploring data

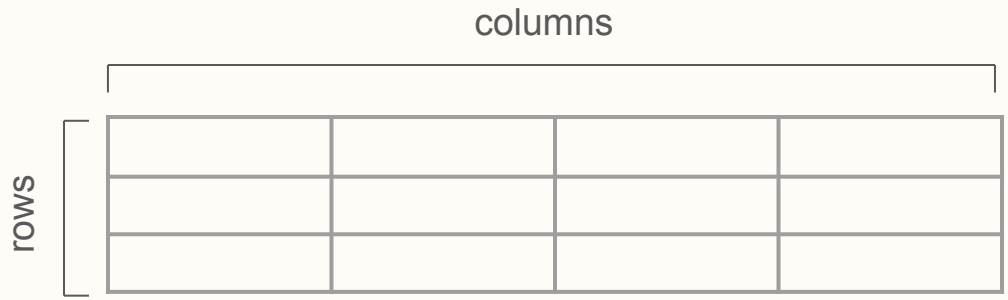
# Panda works with tabular data



## Pandas is Python module designed to handle tabular data

Tabular data refers to data shaped like a table

Data is presented in columns and rows



# Importing module



```
>>> # importing packages
>>> import pandas as pd
```

import as short alias

module vs library vs package

- module - smallest piece of software that is a set of methods and functions
- package - collection of modules
- library - collection of packages and module

# Pandas objects - Series

08

Pandas Series is a one-dimensional array that holds data of any type

A **Series** is like a column in a table

- It is an array of data points of a certain type and has a certain length
- Column has a name

To create a Series

```
mylist = [1, 3, 5]

myseries = pd.Series(mylist)
```

# Pandas objects - Dataframe



Pandas Dataframe is a two-dimensional, size-mutable tabular data

A **Dataframe** is the primary pandas data structure

- Contains labeled rows and columns
- Dictionary-like container for Series objects

To create a Dataframe

```
>>> mydicts = {'col1': [1, 2, 3], 'col2': [4, 5, 6]}
```

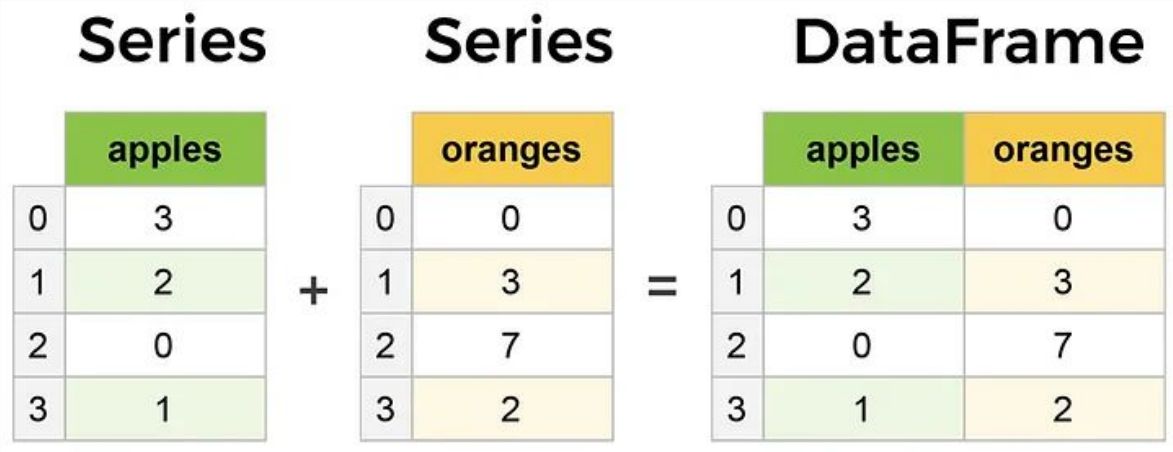
key — list of entries

```
>>> df = pd.DataFrame(data=mydicts)
```

# What type of data does pandas work with?



It’s helpful to think of a DataFrame as a bunch of Series “glued together”



# What type of data does pandas work with?



Assign values to index parameter which will be the row label.

```
>>> df = pd.DataFrame({'col1': [1, 2, 3], 'col2': [4,
5, 6]}, index=["num1", "num2", "num3"])
```

| | col1 | col2 |
|---|---|---|
| num1 | 1 | 4 |
| num2 | 2 | 5 |
| num3 | 3 | 6 |

# Selections and Indexing



row in df - represent data sample

column in df - Variable (can reference index but easier by name)

.iloc selections

iloc index syntax is used to select rows and columns by number

```
df.iloc[<row selection>, <column selection>]
```

*integer list of rows: [0,1, 2]*
*slice of rows: [2:4]*
*single values:1*

*integer list of columns: [0,1, 2]*
*slice of columns: [2:4]*
*single column selections:1*

# .iloc[ ] selection



## .iloc selections

iloc index syntax is used to select rows and columns by integer position

Usage - selecting subsets based on integer index and not index labels

```
df.iloc[<row selection>, <column selection>]
```

```
df.iloc[0]
df.iloc[:, 0]
```

# Class exercise

**CL 11.1**

*What will these loc selection return?*

| | col1 | col2 | col3 |
|---|---|---|---|
| 0 | 1 | 10 | dog |
| 1 | 2 | 12 | cat |
| 2 | 3 | 14 | goat |
| 3 | 4 | 16 | fish |
| 4 | 5 | 18 | cow |

A. df.iloc[0]
B. df.iloc[0, 1:3]
C. df.iloc[:,1]

# .iat[ ] selection

13

## .iat selections

.iat assessor is used to select a single value by its integer position

Usage - optimized for fast access to a single element

```
df.iat[<row selection>, <column selection>]
```

*df.iat[0, 0]*

# .loc[ ] selection



## .loc selections

```
df.loc[<row selection>, <column selection>]
```

*index/label value: ‘mammal’*
*list of labels: [‘mammal’, ‘fungi’]*
*logical/boolean index: df[‘size’] == 100*

*named column: ‘class’*
*list of column names: [‘class’, ‘size’]*
*slice of columns: ‘class’:’species’*

loc index used to
Select row by label/index
Select rows with boolean/conditional lookup

# Class exercise

**CL 11.2**

*What will these loc selection return?*

| | col1 | col2 | col3 |
|---|---|---|---|
| rowA | 1 | 10 | dog |
| rowB | 2 | 12 | cat |
| rowC | 3 | 14 | goat |
| rowD | 4 | 16 | fish |
| rowE | 5 | 18 | cow |

A. df.iloc[“rowB”]]
B. df.loc[:"rowB", "col3"]
C. df.loc["rowC", "col2":]

# Removing NA values



Working with NA (missing values) in DataFrame or Series

```
>>> df.dropna(how="any")
```

`dropna()` usage is to remove missing values from DataFrame or Series

Parameters

- `axis{0, 1}`
- `how{“any”, “all”}`

# Identifying NA values



Working with na (missing values) in DataFrame or Series

```
>>> df.isna()
```

`isna()` usage is to detect missing values from DataFrame or Series. Returns a boolean same-sized object indicating if the values are NA.

# Filling in NA values



Working with na (missing values) in DataFrame or Series

```
>>> df.fillna(value=5)
```

- Method - df.fillna(value=0, method=None)
- df.fillna used to fill in  missing values in a DataFrame or Series with a specified value or method.